PCB 불량 탐지 AI 시스템

Vision AI 기반 이상 탐지 · 컨베이어 벨트 MCU 자동 검사 시스템

팀명

보드가드 BoardGuard

팀장 박수민

팀원 김태환 · 곽혜민 · 이양배

목차

01 **프로젝트 개요**
개발 배경, 문제 정의, 개발 목표

02 **시스템 설계 및 개발환경**
HW · SW 구성과 전체 시스템 아키텍처

03 **핵심 기술 및 구현**
PatchCore 이상탐지, 결함 위치·유형 분석, 벨트 대응 기술

04 **구현 결과 및 검증**
전체 Flow, 구현 시험, 트러블슈팅 및 해결

05 **업무분장**
팀원별 역할과 기여

06 **향후 연구 과제**
시스템 고도화 방향과 후속 개발 주제

07 **프로젝트 수행 후기**
어려웠던 점과 극복 과정, 팀의 소회

01

프로젝트 개요

문제 정의부터 이상탐지 접근까지, 프로젝트의 출발점

주제 및 결과 요약

PCB 불량 탐지 AI 시스템 — 프로젝트 한눈에 보기

주제

Vision AI 기반 PCB 불량 탐지 및 컨베이어 벨트 자동 선별 시스템

핵심 기술

PatchCore 비지도 이상탐지 + 임베딩 기반 결함 유형 분류

핵심 성과

AUROC 1.0 · F1 1.0 (벨트 환경 모델 기준)

결과

박스 기구물 검증 → 벨트 실환경 자동 검사 완성

정상 패턴만으로 학습하는 이상탐지 방식으로 불량 데이터 부족 문제를 해결하고, 실제 컨베이어 환경에서 결함 위치·유형까지 자동 판별하는 시스템 구현

PCB 불량 주제 선정이유

육안으로 판별하기 어려운 결함 다수

1

솔더링 불량

납땜 접합부의 미세한 크랙이나 냉납은 육안 검사에서 놓치기 쉬운 대표적 결함

2

부품 배치 오류

저항·IC 등 소자의 미세한 위치 이탈, 방향 오류는 초기 육안 판별이 어려움

3

크랙 · 미세 균열

기판 표면의 미세 균열은 확산되며 장기적으로 오동작을 유발

기존 방식의 한계와 이상탐지 접근

육안 검사·규칙 기반 AOI·지도학습 모델의 공통적인 구조적 한계

육안 검사

작업자 피로도에 따라 판정 일관성이
떨어지고, 장시간 반복 시 신뢰도 저하

Rule-based AOI

사전에 정의한 규칙 밖의 신규·변형
결함 유형에는 취약하게 대응

일반 지도학습 모델

정상 대비 불량 샘플이 극소수라
학습 데이터 확보가 현실적으로 어려움

충분함

정상 PCB 이미지

동일 조건에서 대량 촬영이 가능해
수백 장 단위로 쉽게 확보

희소함

불량 PCB 이미지

유형이 다양하고 표본이 적어
지도학습용 데이터 확보가 비현실적

결론 정상 패턴만 학습하여 벗어난 것을 탐지하는 One-Class / 비지도 이상탐지 방식

※ AOI: 카메라로 제품을 자동 촬영·검사하는 광학 검사 장비 · One-Class: 정상 데이터만 학습해 이상 여부를 판별하는 학습 방식

개발 목표

위 배경을 바탕으로 세운 두 가지 목표와 최종 완성 시스템

GOAL 1

Vision AI 기반 불량 판별 시스템

PCB 이미지에서 불량 여부를 판별하고,
불량이 있는 경우 그 위치를 특정하며,
불량 원인까지 파악하는 시스템

달성 목표

박스 검증 → 벨트 실환경까지 구현

GOAL 2

AI–MCU 연동 자동 제어 시스템

이상탐지 추론 결과와 MCU 기반
컨베이어 시스템을 연동하여, 정상/비정상
자동 선별 및 원인 분석까지 수행

달성 목표

LED · 모터 제어까지 컨베이어 벨트 연동

02

시스템 설계 및 개발환경

HW · SW 구성부터 전체 파이프라인까지, 시스템 설계

개발 환경 — HW 구성

촬영, 판정, 자동 제어 통합 시스템

1 NVIDIA Jetson

카메라 프레임 캡처 및
TensorRT 실시간 추론

2 IP 카메라

네트워크 스트림 기반
실시간 PCB 영상 획득

3 검사 대상 PCB

HC-SR04
초음파 센서 모듈

4 MCU (M4 보드)

GPIO 인터럽트 기반
Active Object 방식

5 Actuator

스텝 모터로 이송,
서보 모터로 불량 배출

6 상태 표시 LED

정상(녹색) · 비정상(빨간색)
판정 상태 실시간 표시

촬영 기구물: 초기 폼보드 박스 => 최종 컨베이어 벨트 구조로 전환

※ Active Object: 요청을 큐에 쌓아 하나씩 순차 처리하는 비동기 실행 방식

개발 환경 — SW 구성

AI 추론, 판정 로직, 제어, 기록을 구현한 전 영역 소프트웨어

Python · OpenCV

카메라 캡처, 스레드 분리 처리

PatchCore

비지도 이상탐지 모델 학습

ONNX → TensorRT

Jetson 실시간 추론 엔진 변환

resnet18 / wide_resnet50_2

이상탐지 backbone 비교 검증

resnet18 + 1-NN 임베딩

결함 유형 분류 (12종)

SQLite + Web UI

검사 이력 저장 및 실시간 대시보드

GPIO 제어

Jetson ↔ M4 MCU 신호 통신

Event Queue FSM

이벤트 기반 컨베이어 제어 로직

PatchCore(anomalib) 기반 이상탐지, 임베딩 결함 분류, MCU 제어 로직 구현

※ backbone: 이미지 특징을 추출하는 신경망 구조 · 1-NN: 가장 가까운 데이터를 찾아 분류

시스템 아키텍처

카메라 · AI 추론 · 판정 · MCU 제어 · UI 기록 파이프라인

1

카메라 · Jetson

실시간 프레임 캡처
(스레드 분리)

2

**PatchCore
이상탐지 (AI)**

anomaly score /
결함 위치 산출

3

**결함 위치 · 유형
판정**

좌표 영역 매핑,
12종 유형 분류

4

MCU (M4) 제어

LED · 스텝모터 ·
서보모터 구동

5

기록

SQLite 저장,
실시간 대시보드

※ anomaly score: 정상 패턴과 다른 정도를 수치로 나타낸 이상 점수

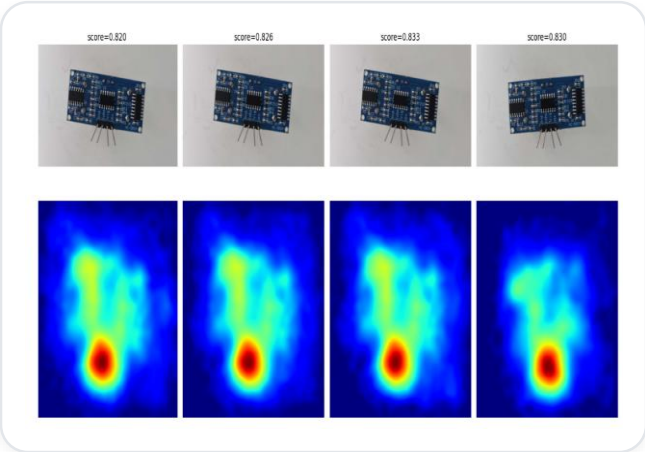
03

핵심 기술 및 구현

PatchCore 이상탐지 및 결함 위치·유형 분석, 벨트 대응 기술

핵심 모델: PatchCore 이상탐지

정상 이미지만으로 학습하는 비지도 이상탐지 알고리즘



PatchCore Anomaly Heatmap 예시

비지도 학습

정상 데이터의 특징(feature)만으로 Memory Bank를 구성

Coreset Sampling

핵심 정상 패치만 선별 저장하여 연산량과 속도를 최적화

Backbone

resnet18 / wide_resnet50_2 비교 검증, 대상 특성에 맞게 선택 적용

Anomaly Map

정상 패턴과의 거리로 픽셀 단위 이상 점수 및 결함 위치 시각화

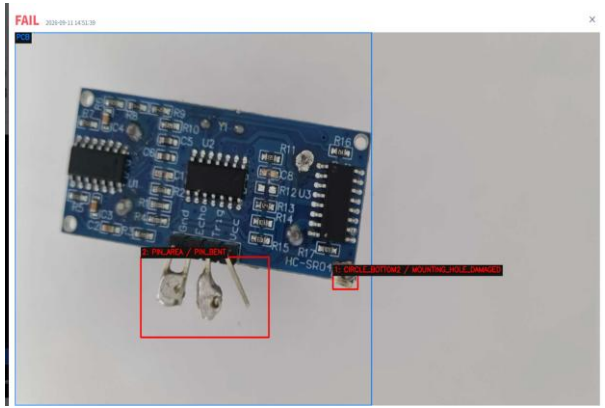
resnet18 / wide_resnet50_2	0.15 ~ 0.2	0.4 ~ 0.5
Backbone	Coreset Sampling Ratio	PASS/FAIL Threshold

※ Coreset Sampling: 대표성이 큰 데이터만 선별해 저장하는 압축 기법 / Memory Bank: 정상 특징을 모아둔 참조 데이터 저장소

결함 위치 매핑

PCB 좌표 영역을 미리 정의하고 결함 위치와 겹치는 영역을 계산해 이름으로 출력하는 방식

1. **1. 좌표 영역 정의**
   정상 PCB 사진 1장을 기준으로 IC 칩, 핀 영역,
   납땜 패드 등 좌표 영역을 JSON으로 정의
2. **2. 결함 박스 겹침 계산**
   PatchCore가 산출한 결함 위치(defect box)가
   어느 정의 영역과 겹치는지 계산
3. **3. 영역명으로 출력**
   겹치는 영역 이름을 PIN_AREA, RESISTOR_AREA1
   처럼 실시간 판정 결과에 함께 표시



실제 판정 화면 — 결함 위치 · 영역 라벨 표시

PIN_AREA / PIN_BENT 처럼 '영역 + 결함 유형'을 함께 출력해 문제 위치를 한눈에 확인 가능

※ JSON: 데이터를 저장·교환할 때 쓰는 텍스트 기반 데이터 형식 · defect box: AI가 결함으로 판단해 표시한 사각 영역

결함 유형 분류

지도학습(YOLO) 대신 임베딩 기반 방식으로 전환한 결함 유형 세분화 판정



한 화면에서 여러 결함 유형을 동시 검출한 예시

resnet18 + 1-NN 임베딩

PCB 클래스별 표본 이미지 대량 확보의 어려움을 고려해 YOLO 지도학습 대신 임베딩 유사도 기반 1-NN 분류 방식으로 전환 — 총 12종 결함 유형 세분화 판정

PIN_BENT	PIN_MISSING	SOLDER_DEFECT
SCRATCH	IC_MISSING	IC_DAMAGED
TX_DEFECT	RX_DEFECT	CRYSTAL_MISSING
CRYSTAL_DAMAGED	RC_MISSING	MOUNTING_HOLE_DAMAGED

총 12종 결함 유형 분류

벨트 환경 대응 판정 · 제어 로직

박스 기구물과 다른 컨베이어 벨트 환경에 맞춰 새로 설계한 판정 로직

1

배경 기준 PCB 존재 검출 (단일 기준 이미지)

빈 벨트 배경 사진 한 장을 기준으로 현재 프레임과 픽셀 차이를 계산해 PCB 진입 여부를 실시간 감지하고, PCB가 없는 경우 AI 추론을 수행하지 않도록 구성

2

다중 프레임 투표 기반 판정 상태머신 (MISSING → INSPECTING → 결과 고정)

PCB 진입 후 60프레임의 PASS/FAIL 판정 결과를 누적하고 다수결로 최종 결과를 확정 → PCB가 이탈할 때까지 결과를 유지하여 프레임별 판정 흔들림 방지

3

MCU 이벤트 큐 기반 상태머신

Jetson의 STOP/PASS/FAIL 신호를 GPIO로 M4에 전달하고, 이벤트 큐 기반 상태머신으로 LED·스텝모터·서보모터를 제어하여 정상/불량 자동 선별

세 기술의 결합으로 컨베이어 환경에서 안정적인 판정과 자동 배출 제어를 구현

※ ROI(Region of Interest): 판정에 사용할 관심 영역

소프트웨어 아키텍처 — 카메라·AI·웹 처리 분리 구조

카메라 수신 · AI 추론 · 웹 서비스를 독립 스레드로 분리해 응답성과 안정성을 확보

1

카메라 스레드
camera_thread()

프레임을 지속 수신하고 연결이 끊기면 자동 재시도. 상태를 live / reconnecting / offline / connecting 4단계로 나눠 대시보드에 실시간 반영

2

추론 스레드
inspection_worker()

카메라 프레임을 받아 앞면·뒷면 PatchCore 모델로 실시간 불량 판정을 수행. 여러 프레임을 모아 결과가 확정되면 그때만 데이터베이스에 저장

3

웹 서비스
Flask REST API

/video_feed 영상 스트리밍과 /inspection 등 검사 API를 통해 대시보드에 실시간 상태 제공

AI 추론이 느려져도 카메라 수신과 웹 서비스까지 같이 멈추지 않도록 각 작업의 실행 흐름을 분리

※ CUDA: GPU 병렬 연산을 위한 NVIDIA 플랫폼

판정 → 제어 연동 — Jetson에서 MCU까지

컨베이어 상태머신과 GPIO 펄스로 판정 결과를 모터·LED 동작까지 연결하는 구조

1
WAIT_ENTRY
PCB가 지정 ROI에 들어올 때까지 대기

2
SETTLING
STOP 신호(GPIO 29) 전송,
0.3초 흔들림 안정화

3
VOTING
60프레임 다수결로 PASS/FAIL 확정

4
GPIO 펄스
PASS(33)/FAIL(31) 전송 →
STM32 EXTI 인터럽트

5
MCU 제어
REJECT_MOVE(스텝 모터) →
REJECT(서보 Push) → RUN

Jetson 컨베이어 상태머신

WAIT_ENTRY → SETTLING → VOTING → WAIT_EXIT

(WAIT_EXIT: PCB가 0.5초 이상 화면을 벗어나야 재판정 허용)

STM32 제어 상태머신

RUN → INSPECT → REJECT_MOVE → REJECT → RUN

(REJECT_MOVE 중엔 새 판정 무시, EVT_MOTOR_DONE만 대기)

PASS → 초록 LED·컨베이어 재가동 / FAIL → 빨강 LED·스텝모터 1300스텝 이동 후 서보로 배출

핵심 기술 — 판정 로직 코드로 보기

60프레임 다수결 판정과 컨베이어 상태 전이의 핵심 로직

```
def state_vote(history):
    counts = Counter(history)
    result, votes = counts.most_common(1)[0]
    if votes >= VOTE_THRESHOLD:
        return result   # PASS 또는 FAIL
    return "INSPECTING"

def check_loop(...):
    if state == WAIT_ENTRY and pcb_in_range():
        send_gpio_pulse(PIN_STOP)  # 29번 핀
        state = SETTLING
    elif state == VOTING:
        result = state_vote(history)
        if result != "INSPECTING":
            send_gpio_pulse(PIN_PASS/FAIL)
            state = WAIT_EXIT
```

※ 실제 소스코드의 핵심 로직을 알아보기 쉽게 축약한 예시 (변수·예외 처리 일부 생략)

다수결 확정

60프레임을 모아 가장 많이 나온 값을 채택,
기준 미달이면 INSPECTING 유지

4단계 상태 전이

WAIT_ENTRY→SETTLING→VOTING→WAIT_EXIT
순서로만 이동해 오판정 방지

GPIO 펄스 전송

판정 즉시 지정 핀으로 신호를 보내
STM32가 곧바로 반응

결과 고정

PCB가 화면에서 완전히 사라져야 다음 판정을
시작해 중복 판정 차단

04

구현 결과 및 검증

전체 시스템 Flow, 구현 시험, 그리고 트러블슈팅과 해결까지

개발 목표 대비 결과

처음 기획한 목표와 실제 구현 결과 비교

GOAL 1

Vision AI 기반 불량 판별 시스템

목표 PCB 이미지에서 불량 여부·위치·원인 판별

결과

- ✓ PatchCore로 벨트 모델 AUROC 1.0 · F1 1.0 달성
- ✓ 결함 위치 좌표 매핑 + 12종 유형 분류 구현
- △ HC-SR04 1개 PCB에서 검증 - · US-015 등 다양한 PCB에서 검증은 미구현
- △ 소자 단위 미세 결함(저항 등)은 미구현 — 카메라 해상도·Jetson 메모리 한계로 보류

GOAL 2

AI–MCU 연동 자동 제어 시스템

목표 이상탐지 결과와 컨베이어 연동, 자동 선별

결과

- ✓ GPIO 펄스로 Jetson–STM32 실시간 연동
- ✓ ROI 상태머신으로 진입·판정·배출 자동화
- ✓ PASS/FAIL별 LED·모터 자동 제어까지 완성

전체 시스템 Flow

컨베이어 벨트 투입부터 자동 선별, 기록까지 사람 개입 없이 동작하는 전 과정

1

PCB 투입 · 이송

컨베이어 벨트로
PCB 이동

2

**카메라 인식
ROI 안정화**

지정 영역 진입 후
연속 프레임 안정화

3

**PatchCore
이상탐지 판정**

PASS / FAIL
anomaly score 산출

4

**결함 위치 · 유형
분석**

좌표 영역 매핑,
12종 유형 분류

5

MCU 제어 + 기록

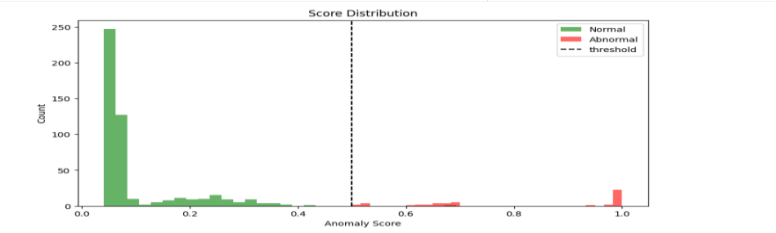
LED·모터 구동,
UI/DB 기록

박스 기구물 검증을 마치고 컨베이어 벨트 실환경에서 완성한 **전 과정 일체형** 파이프라인

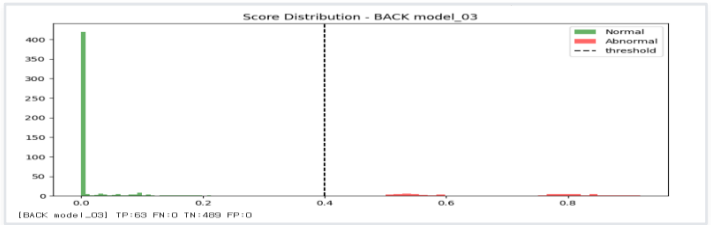
구현 시험 — 최종 모델 성능

resnet_50_2를 backbone으로 한 model_03 채택

모델	학습 데이터	주요 지표	Threshold
model_03_front	벨트 정상 479장 / 불량 50장	AUROC 1.0 / F1 1.0	**0.55**
model_03_back	벨트 정상 489장 / 불량 63장	AUROC 0.999 / F1 1.0	**0.4**



Score Distribution — model_03_front

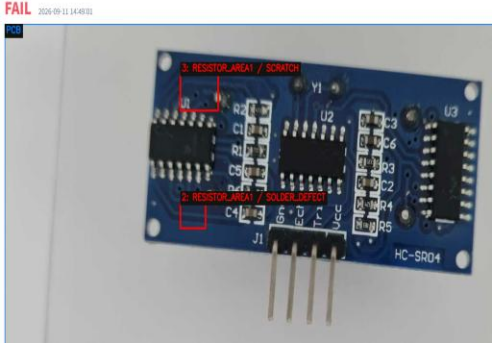


Score Distribution — BELT BACK

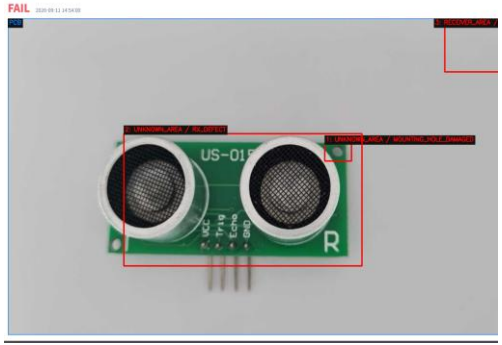
※ AUROC: 정상/불량 구분 성능 지표로 1에 가까울수록 우수 · F1: 정밀도와 재현율의 조화 평균 · Threshold: PASS/FAIL 판별 기준 점수

구현 시험 — 실제 검출 결과

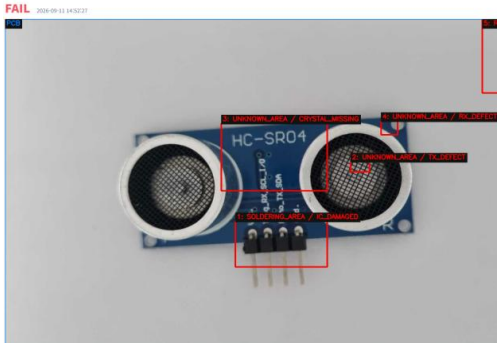
컨베이어 벨트 위에서 결함 위치와 유형을 실시간으로 함께 표시한 결과



RESISTOR_AREA1 — SCRATCH / SOLDER_DEFECT



US-015 — 미학습 모델(불량 판정)



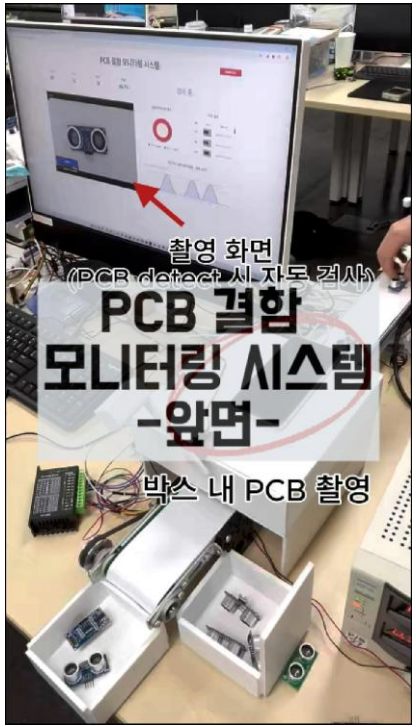
복합 결함 — IC_DAMAGED / CRYSTAL_MISSING 외

다양한 결함 유형에서 안정적으로 동작하는 위치·유형 판정

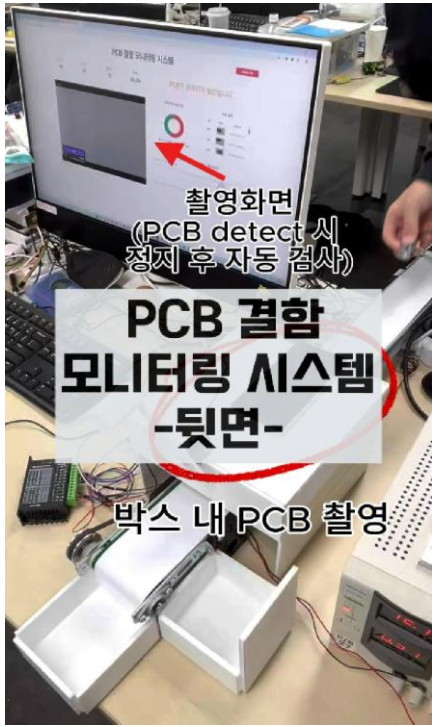
※ 결함 코드(예: SOLDER_DEFECT)는 결함 유형을 영문 약어로 표기한 명칭입니다

실행 영상

컨베이어 벨트 기반 실시간 PCB 검사 및 자동 선별 시연



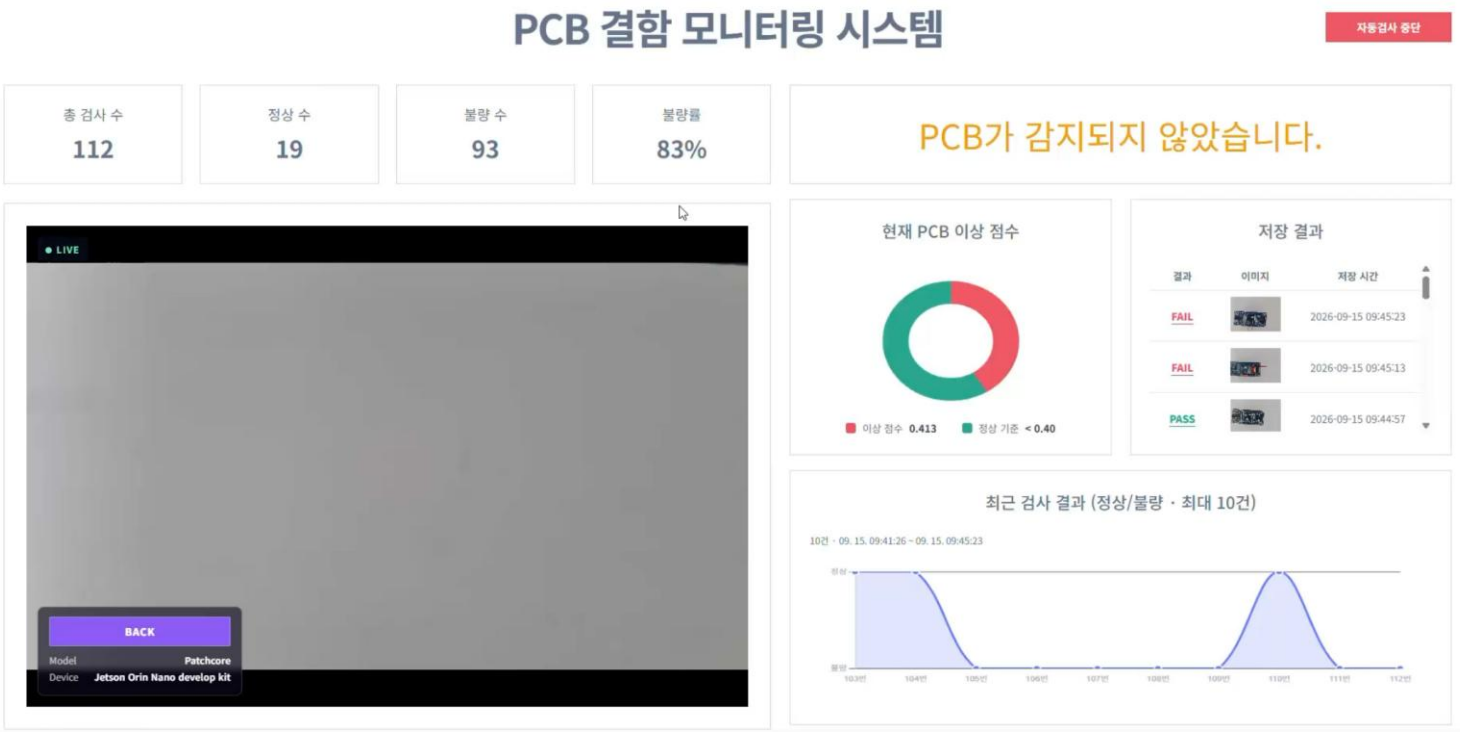
front



back

UI / UX

구현된 관리시스템 UI와 선별 이후 데이터 베이스 저장 과정



트러블슈팅 및 해결

개발 과정에서 마주친 핵심 문제들을 원인 분석을 통해 해결

카메라 프레임 지연

문제
추론이 끝날 때까지 cap.read()를 호출하지 않아 프레임이 버퍼에 계속 누적

해결
별도 스레드로 항상 최신 프레임만 유지(frame_reader)하도록 구조 변경

Jetson–M4 GPIO 노이즈

문제
특정 GPIO 신호 인가 시 인접 핀까지 함께 High로 인식되어 오작동 발생

해결
모터·모터드라이버발 노이즈로 확인, 신호 구간에 10kΩ 저항을 추가하여 해결

PCB 부분 인식 오판정

문제
PCB 일부만 카메라에 들어와도 물체로 인식하여 계속 FAIL 판정이 발생

해결
PCB 박스 중심이 지정 구간에 도달했을 때만 판정하도록 구조 변경

결함 위치 박스 흔들림

문제
매 프레임 독립적으로 threshold를 적용하여 박스 위치·개수가 계속 바뀜

해결
FAIL 판정 순간의 단일 프레임을 캡처하여 결함 위치를 고정 판별

※ frame_reader: 카메라 프레임을 별도로 읽어오는 스레드 · Threshold: PASS/FAIL 판정 기준 점수

결과 분석 및 기대 효과

성능 지표 기반 목표 달성 여부 확인과 기대 효과

목표 달성

AUROC 1.0

정상·불량 구분 성능 지표
(벨트 환경 모델 기준)

목표 달성

F1 1.0

정밀도·재현율의 조화 평균
오탐·미탐 최소화

목표 달성

12종

결함 유형 세분화 분류
원인까지 자동 식별

미달성

PCB 1종 검증

HC-SR04 1개 기종만 검증 — US-015 등
다양한 PCB 확장은 미구현

미달성

소자 단위 결함

저항 등 작은 부품 결함은 미검출 — 카메라 해상도·Jetson 메모리 한계로 보류

기대 효과 실제 생산 라인 적용 시 검사 인력 절감과 24시간 무중단 검사가 가능하며, 카메라·기준 이미지 교체만으로 다른 PCB 모델까지 손쉽게 확장 가능

업무분장

AI · 제어 · UI · 기구물 역할 분배

박

팀장 | 박수민

AI 모델링

PatchCore 모델 학습 및 backbone·해상도 비교 검증, 박스·벨트 환경 데이터셋 구축과 모델 성능 고도화

김

팀원 | 김태환

AI 파이프라인 · 기구물

AI-판정-UI 파이프라인 연동, 카메라 지연 개선, 결함 위치 좌표 영역 정의, 벨트 기구물 제작 및 배경 대응 로직 구현

곽

팀원 | 곽혜민

UI · DB

실시간 판정 UI 설계 및 SQLite DB 연동, PASS/FAIL 검사 이력 아카이브와 이미지 확인 모달 구현

이

팀원 | 이양배

판정 · MCU 제어

판정 알고리즘 및 결함 유형 분류 구현, Jetson–M4 GPIO 연동, 이벤트 큐 기반 상태머신으로 컨베이어 벨트 자동 제어

향후 연구 과제

네 가지 방향의 시스템 고도화 계획

1

실환경 안정화

현장 환경 적응력 강화 (조명·진동 변화에도 오작동 최소화)

2

원인 분석 고도화

불량 원인 정밀 진단 (부품 휨·손상 유형 정밀 분류)

3

경량화·고속화

검사 속도 최적화 (AI 모델 경량화로 속도 향상 & 고정밀 유지)

4

다품종 PCB 확장

다양한 제품(PCB) 호환성 확대 (신규 기판 적용 체계)

네 가지 방향의 고도화로, 실제 양산 라인에 바로 투입 가능한 완성도를 목표로 발전

프로젝트 수행 후기

박수민 팀장

AI 모델링

프로젝트 느낀점

소량의 정상 데이터만으로도 PCB의 이상 여부를 판별할 수 있는 이상 탐지 방식의 가능성을 확인함

어려웠던 점 & 극복

어려움 : 입력 해상도를 높이면 미세 결함 검출은 향상되지만 Jetson의 연산·메모리 제약으로 TensorRT 엔진 변환에 한계 발생

극복 : Backbone과 CoreSet 비율을 조정하며 검출 성능과 추론 부담을 비교해 적절한 모델 구성을 선정

마무리 소회

AI 모델의 성능은 모델 구조뿐 아니라 학습 데이터와 실제 구동 환경의 영향을 크게 받는다는 것을 경험함

김태환 팀원

AI 파이프라인 · 기구물

프로젝트 느낀점

완벽한 모델보다, 환경 변화에 계속 맞춰갈 수 있는 구조가 더 중요하다는 걸 체감함

어려웠던 점 & 극복

어려움 : 여러 부품이 서로 다른 유형의 결함을 동시에 겪었을 때, 위치와 유형을 각각 구분 못 하는 문제

극복 : 결함 박스마다 따로 잘라내어 하나씩 분류하는 방식으로 해결

마무리 소회

작은 선택이 실전에서 결과를 크게 바꿀 수 있다는 것을 경험함

곽혜민 팀원

UI · DB

프로젝트 느낀점

AI·판정·UI가 연결되는 과정에서 각 처리 시점을 명확하게 합의하는 것이 중요함을 체감

어려웠던 점 & 극복

어려움 : 실시간 판정 저장과 화면 갱신을 동시에 처리하는 부분

극복 : SQLite WAL 모드로 읽기·쓰기 충돌 없이 구조를 정리해 해결

마무리 소회

하나의 과정으로 연결하는 경험을 해볼 수 있어 좋았다.

이양배 팀원

판정 · MCU 제어

프로젝트 느낀점

AI의 판정 결과가 GPIO → MCU → 모터 제어까지 이어지는 전체 흐름을 이해함

어려웠던 점 & 극복

어려움 : Jetson에서 하나의 GPIO만 출력했지만 다른 GPIO에도 펄스가 발생해 MCU가 잘못된 이벤트를 인식하는 문제

극복 : 오실로스코프로 GPIO 신호를 직접 확인하고 MCU 입력 신호와 배선 환경을 점검하여 신호 간 간섭 문제를 확인하고 개선

마무리 소회

소프트웨어 로직뿐 아니라 GPIO 신호, 배선, 하드웨어 환경까지 함께 확인해야 문제의 원인을 정확히 찾을 수 있음을 경험함.

Q & A

보드가드 BoardGuard

박수민 · 김태환 · 곽혜민 · 이양배